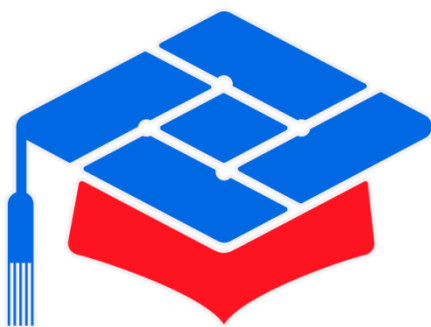




第十届

全国大学生集成电路创新创业大赛

报告类型：_____ 测试方案 _____

参赛杯赛：_____ “信诺达” 企业命题 _____

作品名称：_____ 基于 ST3020 集成电路测试平台的 TP3057 数模混合片的

_____ 自动化测试方案项目设计 _____

队伍编号：_____ CICC1002907 _____

团队名称：_____ 罗德岛 Rose l i a 行动组 _____

目录

1. 项目背景	1
1.1. IC 测试技术在产业研究背景	1
1.2. ATE 分类及市场基本情况	2
1.3. 团队组建与协同介绍	3
2. 测试电路的设计与搭建	5
3. 管脚定义	7
4. 连接性测试	8
5. 参数测试	9
5.1. 上电状态/掉电状态	9
5.2. IIL (Input Low Current)	10
5.3. IIH (Input High Current)	12
5.4. IOZ (Output Current in High Impedance State(TRI-STATE))	13
5.5. ICC0/IBB0 (Power-Down Current)	16
5.6. ICC1/IBB1 (Power-Up Active Current)	18
5.7. GXA (Transmit Gain, Absolute)	20
5.8. GXR (Transmit Gain, Relative to GXA) (300Hz-3000Hz)	24
5.9. GRA (Receive Gain, Absolute)	28
5.10. GRR (Receive Gain, Relative to GRA) (f = 0Hz to 3000Hz)	31
5.11. GRRL (Receive Gain Variations with Level) (-40 dBm0 to +3 dBm0) ..	33
5.12. SFDX (Single Frequency Distortion, Transmit)	37
5.13. SFDR (Single Frequency Distortion, Receive)	42
5.14. IDM (Intermodulation Distortion)	48
6. 功能测试	56
7. AI 工具使用声明	57
8. 参考文献	58

基于 ST3020 集成电路测试平台的 TP3057 数模混合片的自动化测试方案项目设计

1. 项目背景

1.1. IC 测试技术在产业研究背景

1.1.1. 技术概述

集成电路测试是贯穿芯片“设计-制造-封测-应用”全生命周期的关键环节，指通过专业测试设备与技术对芯片进行性能、功能、可靠性进行检验的一系列活动，其核心目的是筛选不合格产品、保障芯片良率与可靠性。按测试目的与阶段划分，芯片测试可分为晶圆测试 (Chip Probing,)、成品测试 (Final Test,)、系统级测试 (System Level Test)、可靠性测试等。

现代 IC 测试高度依赖自动测试设备 (ATE)，它集成了精密测量单元、向量发生器和高速数字通道，可以对芯片进行参数测试、功能测试、故障检测、可靠性测试等等。一台完整的 ATE 系统主要由测试主机（提供电源、时钟与数字/模拟信号）、测试界面（自动搬运与定位芯片的 Handler/Prober）、测试程序（定义测试项目与判定条件）和测试治具（Load Board/Probe Card，实体连接芯片与测试系统的介面）等部分构成，与分选机（Handler）或探针台（Prober）等设备共同构成了自动化测试系统。

1.1.2. 产业现状

随着消费电子迭代、汽车电子与 AI 算力芯片需求爆发，全球集成电路测试行业市场规模持续增长。2025 年全球半导体测试服务市场规模约 140.7 亿美元，预计到 2032 年将增长至 227.5 亿美元，年均复合增长率 (CAGR) 约为 6.9%^[1]。市场增长主要归功于 5G 通信、高性能计算与自动驾驶汽车的推进，以及半导体器件日益复杂的设计、3D 堆叠和小芯片封装带来的测试难度提升^[2]。行业集中度依旧较高，泰瑞达、爱德万、科休等传统 ATE 巨头长期占据市场主导地位，但新兴厂商正积极切入细分领域展开竞争。

中国 IC 测试产业受益于本土半导体国产替代需求与庞大下游市场的双重拉动，近年来发展势头十分迅猛。2025 年中国自动化测试设备市场规模已达 323.24 亿元人民币，预计到 2032 年有望突破 609 亿元^[3]。在第三方检测服务领域，中国市场规模 2025 年预计达 125.5 亿元，2023 至 2025 年复合增长率约 13%至 15%，增长动力主要来自先进封装检测需求的爆发以及车规级芯片验证的强制要求。竞争格局方面，形成了“封测一体巨头”与“独立第三方专业测试”并存的生态。以长电科技、通富微电、华天科技为代表的封测巨头，凭借技术和资本积累，在先进封装及测试一体化方案上优势突出，2024 年前三者营收分别达 359.62 亿元、238.82 亿元和 144.62 亿元；而以伟测科技、利扬芯片、华岭股份为代表的本土独立第三方专业测试厂商，专注于晶圆测试和成品测试，凭借定制化、中立性等优势正快速抢占细分市场。在测试设备端，以华峰测控、长川科技、悦芯科技为代表的国内企业已实现在模拟及数模混合测试机领域的规模国产化，并正逐步向 SoC、存储及高端 ATE 市场渗透，检测设备国产器件自给率稳步提升。展望未来，Chiplet 和 3D 先进封装技术的普及正推动封装与测试的边界日益模糊，系统级测试和异构集成测试的重要性愈发凸显。在政策扶持与市场需求共振之下，国内 IC 测试产业有望加速从产能扩张走向技术与价值重塑，在全球半导体测试产业版图中扮演更加关键的角色。

1.2. ATE 分类及市场基本情况

1.2.1. ATE 的基本概念

自动测试设备（ATE——Automatic Test Equipment）是指向半导体器件提供电气信号并将输出信号与期待值进行比较、以检验其是否按照设计运行的电子测试系统。ATE 系统以最少的人工干预对芯片进行性能、质量、功能和可靠性测试，将被测试的芯片称为 DUT（Device Under Test）。ATE 的主要功能包括向芯片施加激励信号、采集芯片响应信号、与预设标准参数进行比对判定，从而实现芯片的功能验证、性能测试和可靠性考核。

在半导体封测流程中，ATE 贯穿晶圆测试（CP 测试）和成品测试（FT 测试）两个核心环节，与探针台（Prober）和分选机（Handler）等辅助设备协同

作业，构成一体化测试系统。测试机是整个 ATE 系统的核心，其性能和精度直接影响芯片筛选的质量和效率；探针台与分选机则分别用于测试过程中晶圆/芯片的物理定位、传输与分选。

根据测试芯片类型的不同，ATE 可划分为存储测试机、SoC（系统级芯片）测试机、模拟/混合类测试机和射频测试机四大细分领域

1.2.2. 全球 ATE 市场规模

近年来，随着全球集成电路测试行业市场规模持续增长，全球自动化测试设备（ATE）市场规模也在稳健增长。据 Fortune Business Insights 统计，2025 年全球自动化测试设备市场规模为 54.1 亿美元，预计将从 2026 年的 56.3 亿美元增长到 2034 年的 97.2 亿美元，预测期内复合年增长率为 7.10%。2025 年，亚太地区以 50.70% 的份额主导全球市场^[4]。

中国市场方面，受益于国产替代浪潮和全球最大半导体消费市场的双重驱动，国内 ATE 市场正加速发展。据贝哲斯咨询数据，2025 年中国半导体自动测试设备市场规模已达约 103.24 亿元人民币^[5]。在测试机市场中，SoC 和存储测试机因技术壁垒最高，合计占据约 80% 的市场份额。国产替代呈现明显的结构性特征：模拟与分立器件测试机国产化率已达到约 80%，但 SoC 测试机国产化率仅约 10%，存储测试机更是低至约 8%，后两者的替代空间十分广阔^[6]。以华峰测控、长川科技、悦芯科技等为代表的国内厂商正加速技术突破。华峰测控已从模拟领域切入 SoC 测试，其面向 AI 和高性能计算的 STS8600 新一代测试平台正处于放量前夕；悦芯科技则围绕 SoC、存储器和功率芯片打造了完整的 ATE 测试系统矩阵，其 T800 系列和 TM8000 系列分别覆盖 SoC 及各类存储器测试需求^[7]。

1.3. 团队组建与协同介绍

本项目团队由两人组建，分工明确又紧密协作。在项目启动阶段，我们依据各自的特长与任务需求进行了合理分工：一名成员主要负责连接性测试和参数测试方案与代码的撰写以及文档整理，另一名成员主要负责功能测试的方案

与代码撰写、接口板设计以及图形文件的编制。双方在各自牵头任务的同时，始终保持密切沟通与相互支持，确保设计与文档工作协调并进。

在文档撰写过程中，我们特别注重测试方案的科学性与专业性，围绕方案的合理性进行了多轮深入讨论，并及时对讨论的结论进行系统归纳与整理。基于整理出的测试方案需求，负责硬件设计的成员进一步完成了接口板的详细设计及配套图形文件的编写。随后，我们以方案和设计材料为依据，结合 ATE 系统手册，进行测试程序的开发。

在整个设计与编程过程中，团队成员持续保持协作，对出现的问题及时共享、共同分析，迅速进行定位与更正，从而有效保障了最终成果的质量和一致性。

2. 测试电路的设计与搭建

在测试电路的设计与搭建过程中，本方案严格遵循 TP3057 芯片手册中应用信息部分的指导，并结合典型电路架构，依据 ATE 设备 ST3020 的量程特性及实际测试需求，对负载配置与信号调理进行了适配性设计。最终电路方案如下：

电源去耦方面，依据手册建议，在 VCC 与 VBB 电源引脚靠近芯片处各并联一枚 $0.1\ \mu\text{F}$ ($100\ \text{nF}$) 高频去耦电容，以提供低阻抗退耦路径。电容材质选用 X7R 多层陶瓷电容，布局紧贴电源引脚，最小化寄生电感。芯片手册同时建议在星形接地公共点处对 VCC 和 VBB 额外配置 $10\ \mu\text{F}$ 储能电容。考虑到本测试中 ATE 器件电压源 (DPSB) 直接为芯片供电，其输出端已具备毫欧级低阻抗且板上无需额外储能即可满足瞬态电流需求，故未部署该 $10\ \mu\text{F}$ 电容，以简化 BOM 并避免板上大电容对供电快速上下电响应的影响。

负载适配方面，芯片的模拟输出引脚在测试中端接 $600\ \Omega$ 电阻负载，这是 TP3057 手册规定的交流特性测试条件，用于模拟典型应用负载。被测芯片 TP3057 接收输出在 $600\ \Omega$ 负载下的峰值电压为 $\pm 2.5\text{V}$ (约 $1.77\text{V}_{\text{rms}}$)，而 ST3020 音频电压表(AVM)的 4V_{rms} 测量档位在 $900\text{Hz}\sim 20\text{kHz}$ 频率范围内具有 $\pm(0.5\%+1.5\text{mV})$ 的测量准确度，信号幅度完全落在量程覆盖范围内且保留足够余量，因此未配置衰减电路。省去电阻分压网络不仅简化了信号链路，也避免了分压电阻的附加噪声、带宽受限及对输出阻抗匹配的影响，使测量路径保持最小信号损失。

针对模拟引脚多模式测试需切换资源的需求，选用了 SRD-05VDC-SL-C 型功率继电器实现测试电路切换控制。继电器触点用于切换被测通道、选择不同激励源，以及将待测引脚短接或与激励源连接，以适应各类参数测试。控制线圈由 ATE 内部继电器驱动信号直接控制，测试程序依据该继电器技术手册中的吸合与释放时间参数设置操作延时，让测量发生在触点稳定闭合之后，有效避开了弹跳及瞬态过程对模拟测量精度的干扰。

PCB 布局与布线部分，综合考虑后走线优先级依次确定为：数字信号线、模拟信号线、继电器控制线、电源线。数字引脚在接收图形激励时，通道间时

序偏移要求控制在 ns 级以内。鉴于数字激励速率为 2.048 MHz，属于低速信号，且走线总长有限，设计中采用等长、等间距的类差分走线策略。虽然没有进行 100 Ω 差分阻抗控制，但紧耦合平行布线配合连续参考平面，使信号与回流路径包围的环路面积最小化，有效降低了共模噪声和串扰，保障了信号边沿的单调性及通道间的时序一致性。

地线处理方面，严格遵循芯片手册建议的星形接地策略：芯片的 GNDA 引脚作为模拟地的唯一参考点，各功能模块的地线以星形方式汇聚至 GNDA 引脚附近的公共接地点，避免共用地总线引入的相互干扰。以该公共接地点为中心，通过大面积铺铜构造低阻抗回流路径。在此基础上，为进一步降低地平面在芯片内部敏感电路下方引入寄生耦合的风险，所有布线层在芯片底座正下方区域均实施了禁地处理，禁地形状和尺寸与芯片底座轮廓一致。双层板以底层作为相对完整的参考地层，关键信号优先在顶层布线，在控制成本的同时兼顾了信号回流路径的连续性和电磁兼容性。此外，继电器线圈的 5V 驱动回路在 PCB 布局上独立走地，其回流路径与芯片 GNDA 的星形接地点分离，避免线圈开关瞬态电流通过共地阻抗耦合至模拟信号链路，造成测量本底噪声抬升。

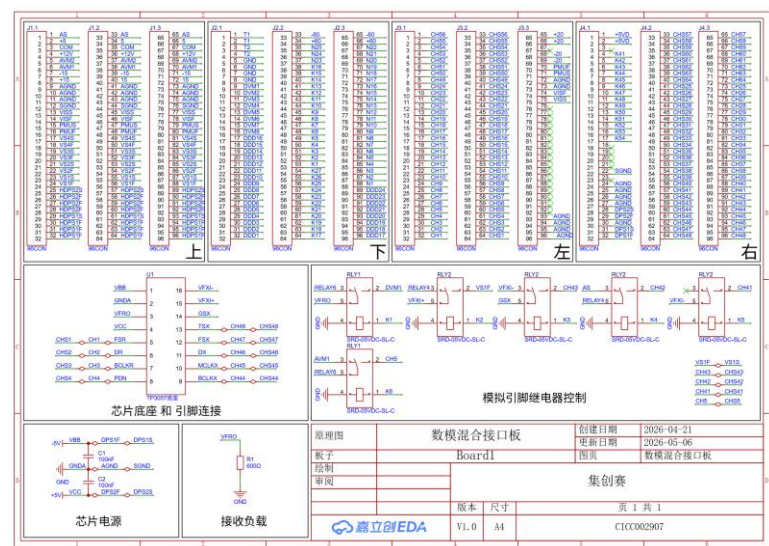


图 2.1-电路原理

电路原理图见图 2.1-电路原理（见附件“电路原理.jpg”）。

3. 管脚定义

见表 3.1-管脚定义 与 图 3.1-引脚连接

表 3.1-管脚定义

芯片引脚	通道号
TSX	48
MCLKX	45
BCLKX	44
PDN	4
BCLKR	3
FSX	47
DX	46
FSR	1
DR	2

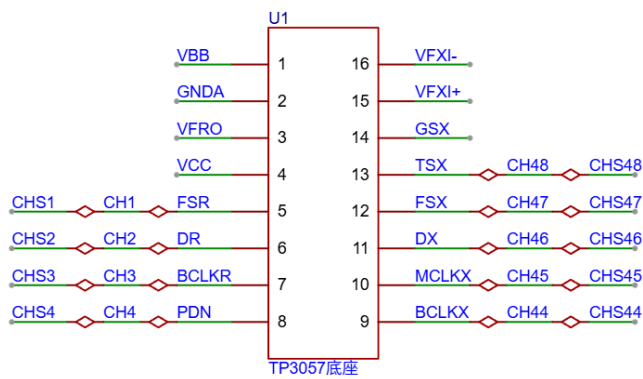


图 3.1-引脚连接

4. 连接性测试

- 1) **测试意义：**验证物理连接是否正确、引脚间是否存在短路，验证芯片内部保护二极管是否正常，提高测试效率
- 2) **测试原理：**利用芯片每个引脚内部集成有对 VCC 和 GND 的保护二极管，通过施加小电流（例如 $\pm 100\mu\text{A}$ ）并测量引脚电压，根据二极管正向导通电压降（约 0.4V~0.9V）来判断引脚与电源/地之间的连接是否正常
- 3) **测试条件：**对管脚施加 $-100\mu\text{A}$ 电流，待测管脚与地间的二极管电压范围应-0.1~-1.9V 之间
- 4) **被测引脚：**除 GNDA、VCC、VBB 外所有管脚
- 5) **测试步骤：**所有管脚接地，闭合继电器，将被测器件管脚接到 ATE 的 PMU，由 PMU 施加一个 $-100\mu\text{A}$ 的电流，测量测试管脚与地间的二极管电压，电压范围应在-0.1~-1.9V 之间。测量结束后打开继电器，以待后续测试使用
- 6) **使用的 ATE 资源**
PMU
- 7) **参考代码**（具体代码见“程序代码.txt”）

```
//CON-TEST
CLOSE_RELAY("3,4,5,6"); //闭合继电器，使模拟脚连接 pmu
SET_DPS(1,0,V,20,MA); //选用 1 号电源通道 DPS1，将 VCC 电压设为 0
SET_DPS(2,0,V,20,MA); //选用 2 号电源通道 DPS2，将 VBB 电压设为 0
PMU_CONDITIONS(FIMV,-0.1,MA,2,V); //加流测压，施加电流为-0.1 mA，电
压箝位值设为 2 V
if(!PMU_MEASURE("1,2,3,4,5,44,45,46,47,48",20,"CON_",V,-0.1,-1.9))
//依次测量适配器 1-5，44-48 通道的电压，判断是否在-0.1 V 至-1.9 V 之间
CLEAR_RELAY("3,4,5,6"); //打开继电器
BIN(1);
Delay(10);
```

5. 参数测试

5.1. 上电状态/掉电状态

在参数与功能测试中，需要视情况将 TP3057 置于上电/掉电状态中。

5.1.1. 上电状态 (Power-Up Mode)

1) 进入条件

电源电压： $VCC = +5V$, $VBB = -5V$, $GNDA = 0V$

控制引脚：MCLKR/PDN 引脚连续为低电平（通常接 GND），或提供有效的接收主时钟（1.536/1.544/2.048 MHz）

时钟与帧同步：根据功能需要，提供 MCLKX/BCLKX/FSX（发送）或 MCLKR/BCLKR/FSR（接收）时钟。即使部分时钟不提供，芯片仍处于上电状态，但相应模块不工作

2) 电气特征

全功能工作：编码器、解码器、滤波器、参考电压源、数字接口均正常，所有 I/O 引脚（DX、DR、FSX、FSR、BCLKX、BCLKR、MCLKX、MCLKR、VF1+、VF1-、GSX、TSX）按数据手册规范工作

典型功耗：50 mW ($VCC = +5V$, $VBB = -5V$, $T_A = 25^{\circ}C$)

3) 参考图形（具体图形见“向量图形.txt”）

索引号 0 号——将芯片置于上电状态：MCLKX 接 2.048 MHz 方波，BCLKX 接 2.048 MHz 方波，MCLKR/PDN 接 GND，FSX 接 GND

5.1.2. 掉电状态 (Power-Down Mode)

1) 进入条件：

自动掉电：当首次施加电源时，上电复位电路会初始化 COMBO 器件，并将其置于掉电状态

手动掉电：有以下两种方法可使 TP3057 进入掉电状态

(1) 将 MCLKR/PDN 引脚连续接高电平

(2) 将 FSX 和 FSR 输入连续低电平，器件将在最后一个 FSX 或 FSR 脉冲后约 1 ms 自动进入掉电状态

2) 电气特征:

内部大部分电路关闭，仅保留极小偏置电路。

典型功耗: 3 mW ($V_{CC}=5V$, $V_{BB}=-5V$, $T_A=25^{\circ}C$)。

数字接口状态: DX 输出为高阻态 (TRI-STATE)，不受 FSX 控制。DR、FSX、FSR、BCLKX、BCLKR、MCLKX 等输入引脚无效。

3) 退出掉电模式:

将 MCLKR/PDN 引脚拉低即可恢复正常工作；需重新稳定时钟（建议等待 $10\mu s \sim 100\mu s$ 后开始正常操作）。

4) 参考图形（具体图形见“向量图形.txt”）

索引号 1 号——使芯片进入掉电状态：将 MCLKR/PDN 接 VCC (+5V)

5.2. IIL (Input Low Current)

1) 参数含义: Input High Current: 数字输入引脚被置于输入低电平区间 ($GND \leq V_{IN} \leq V_{IL}$) 时，流入/流出引脚的直流漏电流。

2) 测试类别: 直流参数

3) 测试原理: 在规定的低电平电压下，测量从芯片引脚流出的漏电流。

4) 测试条件: $GND \leq V_{IN} \leq V_{IL}$ ($V_{ILmax} = 0.6 V$)

5) 被测引脚: 所有数字输入引脚

MCLKR/PDN、MCLKX、BCLKX、BCLKR/CLKSEL、FSX、FSR、DR、CLKSEL 等全部数字输入

6) 测试步骤

i) 芯片供电与接地

接 GNDA 到系统地

接 VCC (+5V)、VBB (-5V)，等待电源稳定

保持芯片处于上电模式

运行图形文件使所有输入端为高电平

ii) 对单个数字输入引脚施加低电平

设置 PMU 测试模式为加压测流，被测管脚施加 0.4V 电压，箝位电流
设为 100 μ A

iii) 测量并判断输入管脚的电流

测量输入管脚的电流，判断其是否满足： $-10 \mu\text{A} \leq I_{IL} \leq +10 \mu\text{A}$

7) 使用的 ATE 资源

PMU

8) 参考代码（具体代码见“程序代码.txt”）

```
void SETUP(void)
{
    SET_DPS(DPS1, 5.0, V, 100, MA);
    SET_DPS(DPS2, -5.0, V, 100, MA);
    Delay(10);
    SET_PERIOD(488);
    SET_TIMING(100, 350, 450);
    SET_INPUT_LEVEL(2.2, 0.6);
    SET_OUTPUT_LEVEL(2.4, 0.4);
    FORMAT(NRZ0, "48,4,47,46,1,2"); // FSR-1;DR-2;BCLKR-3;PDN-
4;BCLKX-44;MCLKX-45;DX-46; FSX-47; TSX-48
    FORMAT(RZ, "45,44,3");
    RUN_PATTERN(3, 1, 0, 0); // 运行图形 3，产生 FSX 脉冲，使芯片上电
    Delay(10);
}

void PASCAL TP3057()
{
    //系统初始化（为后续图形运行做准备）
    CLEAR_ALL();

    //IIL-TEST
    SETUP();
    RUN_PATTERN(1,1,0,0); //运行 1 号图形文件，使所有输入端置“1”
    PMU_CONDITIONS(FVMI,0.4,V,100,UA); //加压测流
    if(!PMU_MEASURE("1,2,3,4,44,45,47,48",5,"IIL",UA,10,-10)) //测量并
判断输入管脚的电流
    BIN(2);
}
```

9) 参考图形（具体图形见“向量图形.txt”）

索引号 2 号——使所有数字输入端置“1”

5.3. I_{IH} (Input High Current)

1) 参数含义: Input High Current: 数字输入引脚被置于输入高电平区间 ($V_{IH} \leq V_{IN} \leq V_{CC}$) 时, 流入/流出引脚的直流漏电流。

2) 测试类别: 直流参数

3) 测试原理: 在规定的高电平电压下, 测量从芯片引脚流出的漏电流。

4) 测试条件: $V_{IH} \leq V_{IN} \leq V_{CC}$ ($V_{IHmin} = 2.2\text{ V}$)

5) 被测引脚: 所有数字输入引脚

MCLKR/PDN、MCLKX、BCLKX、BCLKR/CLKSEL、FSX、FSR、DR、
CLKSEL 全部数字输入

6) 测试步骤

i) 芯片供电与接地

接 GNDA 到系统地

接 VCC (+5V)、VBB (-5V), 等待电源稳定

使芯片保持在上电状态

运行图形文件使所有输入端为高电平

ii) 对单个数字输入引脚施加低电平

设置 PMU 测试模式为加压测流, 被测管脚施加 3V 电压, 箝位电流设为 $100\text{ }\mu\text{A}$

iii) 测量并判断输入管脚的电流

测量输入管脚的电流, 判断其是否满足: $-10\text{ }\mu\text{A} \leq I_{IH} \leq +10\text{ }\mu\text{A}$

7) 使用的 ATE 资源

PMU

8) 参考代码（具体代码见“程序代码.txt”）

```
void SETUP(void)
{
```

```

    SET_DPS(DPS1, 5.0, V, 100, MA);
    SET_DPS(DPS2, -5.0, V, 100, MA);
    Delay(10);
    SET_PERIOD(488);
    SET_TIMING(100, 350, 450);
    SET_INPUT_LEVEL(2.2, 0.6);
    SET_OUTPUT_LEVEL(2.4, 0.4);
    FORMAT(NRZ0, "48,4,47,46,1,2"); // FSR-1;DR-2;BCLKR-3;PDN-4;BCLKX-
44;MCLKX-45;DX-46; FSX-47; TSX-48
    FORMAT(RZ, "45,44,3");
    RUN_PATTERN(3, 1, 0, 0); // 运行图形 3，产生 FSX 脉冲，使芯片上电
    Delay(10);
}

void PASCAL TP3057()
{
    //系统初始化（为后续图形运行做准备）
    CLEAR_ALL();

    //IIL-TEST
    SETUP();
    RUN_PATTERN(1,1,0,0); //运行 1 号图形文件，使所有输入端置“1”
    PMU_CONDITIONS(FVMI,0.4,V,100,UA); //加压测流
    if(!PMU_MEASURE("1,2,3,4,44,45,47,48",5,"IIL",UA,10,-10)) //测量并判
断输入管脚的电流
    BIN(2);
}

```

9) 参考图形（具体图形见“向量图形.txt”）

索引号 3 号——使所有数字输入端置“0”

5.4. IOZ (Output Current in High Impedance State(TRI- STATE))

- 1) 参数含义：Dx 引脚处于三态高阻态时，在 $0V \leq V_O \leq V_{CC}$ 电压下的流入 / 流出漏电流
- 2) 测试类别：直流参数

3) **测试原理：**在芯片的输出引脚施加电压，测量其在高阻态下从引脚流出的漏电流。

4) **测试条件：**DX 处于三态高阻态, $GNDA < VO < VCC$

被测引脚：D_x

5) **测试步骤**

i) **芯片供电与接地**

接 GNDA 到系统地

接 VCC (+5V)、VBB (-5V)，等待电源稳定

使芯片保持在上电状态

使 FSR、DR、BCLKR/CLKSEL 接 GND

VFXI+、VFXI-、GSX、VFRO、TSX 开路（模拟部分/开漏输出，不影响 DX）

ii) **使 D_x 进入三态高阻**

MCLKX 接 2.048 MHz 方波，BCLKX 接 2.048 MHz 方波，

MCLKR/PDN 接 GND，FSX 接 GND

iii) **施加测试电压并测量对 VCC 漏电 IOZL**

设置 PMU 测试模式为加压测流，被测管脚施加 0V 电压，箝位电流设为 100 μA，读取电流值并判断是否满足 $-10 \mu A \leq IOZL \leq +10 \mu A$

iv) **施加测试电压并测量对 GND 漏电 IOZH**

设置 PMU 测试模式为加压测流，被测管脚施加 5V 电压，箝位电流设为 100 μA，读取电流值并判断是否满足 $-10 \mu A \leq IOZH \leq +10 \mu A$

6) **使用的 ATE 资源**

PMU

7) **参考代码**（具体代码见“程序代码.txt”）

```
#include "StdAfx.h"
#include "testdef.h"
#include "data.h"
#include "math.h"

void SETUP(void)
{
```



```

    SET_DPS(DPS1, 5.0, V, 100, MA);
    SET_DPS(DPS2, -5.0, V, 100, MA);
    Delay(10);
    SET_PERIOD(488);
    SET_TIMING(100, 350, 450);
    SET_INPUT_LEVEL(2.2, 0.6);
    SET_OUTPUT_LEVEL(2.4, 0.4);
    FORMAT(NRZ0, "48,4,47,46,1,2"); // FSR-1;DR-2;BCLKR-3;PDN-
4;BCLKX-44;MCLKX-45;DX-46; FSX-47; TSX-48
    FORMAT(RZ, "45,44,3");
    RUN_PATTERN(3, 1, 0, 0); // 运行图形 3，产生 FSX 脉冲，使芯片上电
    Delay(10);
}

void PASCAL TP3057()
{
    //系统初始化（为后续图形运行做准备）
    CLEAR_ALL();

    //IOZ-TEST
    SETUP();
    RUN_PATTERN(4,1,0,0); //运行 4 号图形文件，使 Dx 进入三态高阻
    PMU_CONDITIONS(FVMI,0,V,100,UA); //加压测流，测 IOZL（对 Vcc 漏电）
    if(!PMU_MEASURE("46",5,"IOZL",UA,10,-10)) //测量并判断 Dx 的电流
    BIN(4);
    PMU_CONDITIONS(FVMI,5,V,100,UA); //加压测流，测 IOZH（对 gnd 漏电）
    if(!PMU_MEASURE("46",5,"IOZH",UA,10,-10)) //测量并判断 Dx 的电流
    BIN(4);
}

```

8) 参考图形（具体图形见“向量图形.txt”）

4 号——使 Dx 进入三态高阻：MCLKX、BCLKX 接 2.048 MHz 方波，MCLKR/PDN 接 GND，FSX 接 GND，FSR、DR、BCLKR/CLKSEL 接 GND

5.5. ICC0/IBB0 (Power-Down Current)

- 1) **参数含义：**掉电模式下 VCC / VBB 电源的无负载电流
- 2) **测试类别：**直流参数
- 3) **测试原理：**让 TP3057 进入低功耗的“掉电模式”，然后测量 VCC / VBB 电源的输入电流。其目的是验证芯片在待机状态下功耗是否正常，以剔除有漏电缺陷的芯片。
- 4) **测试条件：**无负载，需在芯片先完成上电后再测量
- 5) **被测引脚：**VCC / VBB
- 6) **测试步骤**

i) 芯片供电与接地

接 GNDA 到系统地

接 VCC (+5V)、VBB (-5V)，等待电源稳定

使芯片保持在上电状态

ii) 使芯片进入掉电模式

将 MCLKR/PDN 接 VCC (+5V)，让芯片进入掉电模式使所有输出端 (VFRO、DX、TSX、GSX) 悬空 (无负载)，VFXI+、VFXI-、FSX、FSR、DR、BCLKR/CLKSEL、BCLKX、MCLKX 接地

等待 10ms，使芯片稳定

iii) 测量 ICC0 并判断

使用 DPS 直接测量电流值 ICC0，判断是否满足 $ICC0 < 1.5MA$

iv) 测量 IBB0 并判断

使用 DPS 直接测量电流值 IBB0，判断是否满足 $IBB0 < 0.3MA$

7) 使用的 ATE 资源

DPS

8) 参考代码 (具体代码见“程序代码.txt”)

```
#include "StdAfx.h"
#include "testdef.h"
#include "data.h"
```

```

#include "math.h"

void SETUP(void)
{
    SET_DPS(DPS1, 5.0, V, 100, MA);
    SET_DPS(DPS2, -5.0, V, 100, MA);
    Delay(10);
    SET_PERIOD(488);
    SET_TIMING(100, 350, 450);
    SET_INPUT_LEVEL(2.2, 0.6);
    SET_OUTPUT_LEVEL(2.4, 0.4);
    FORMAT(NRZ0, "48,4,47,46,1,2"); // FSR-1;DR-2;BCLKR-3;PDN-4;BCLKX-
44;MCLKX-45;DX-46; FSX-47; TSX-48
    FORMAT(RZ, "45,44,3");
    RUN_PATTERN(3, 1, 0, 0); // 运行图形 3, 产生 FSX 脉冲, 使芯片上电
    Delay(10);
}

void PASCAL TP3057()
{
    //系统初始化（为后续图形运行做准备）
    CLEAR_ALL();

    //ICC0IBB0-TEST
    SETUP();
    RUN_PATTERN(5,1,0,0); //使芯片进入掉电状态, 测量 ICC0IBB0
    Delay(50);
    if(!DPS_MEASURE(DPS1, R20MA, 5, "ICC0",MA, 1.5, No_LoLimit)) {//这里
量程也可以使用 2ma
        BIN(5);
    }
    if(!DPS_MEASURE(DPS2, R2MA, 5, "IBB0",MA, 0.3, No_LoLimit)) {
        BIN(5);
    }
}
}
}

```

9) 参考图形（具体图形见附录）

索引号 5 号——使芯片进入掉电状态, 测量 ICC0IBB0: 将 MCLKR/PDN 接 VCC (+5V), 让芯片进入掉电模式使所有输出端 (VFRO、DX、TSX、GSX) 悬空 (无负载), VFXI+、VFXI-、FSX、FSR、DR、BCLKR/CLKSEL、BCLKX、MCLKX 接地

5.6. ICC1/IBB1 (Power-Up Active Current)

- 1) **参数含义：**芯片在上电状态（Power-Up Active）下的正/负电源（VCC / VBB）电流。
- 2) **测试类别：**直流参数
- 3) **测试原理：**让 TP3057 进入上电激活状态，然后测量 VCC / VBB 电源的输入电流。
- 4) **测试条件：**无负载
- 5) **被测引脚：**VCC / VBB
- 6) **测试步骤**

i) 芯片供电与接地

接 GNDA 到系统地

接 VCC (+5V)、VBB (-5V)，等待电源稳定

使芯片保持在上电状态（MCLKX 接 2.048 MHz 方波，BCLKX 接 2.048 MHz 方波，MCLKR/PDN 接 GND，FSX 接 8kHz 脉冲）

使所有输出端（VFRO、DX、TSX、GSX）悬空（无负载）

VFXI+、VFXI-、FSX、FSR、DR、BCLKR/CLKSEL 接地

ii) 测量 ICC1 并判断

使用 DPS 直接测量电流值 ICC0，判断是否满足 $ICC1 < 9MA$

iii) 测量 IBB1 并判断

使用 DPS 直接测量电流值 IBB0，判断是否满足 $IBB1 < 9MA$

7) 使用的 ATE 资源

DPS

8) 参考代码（具体代码见“程序代码.txt”）

```
#include "StdAfx.h"
#include "testdef.h"
#include "data.h"
```

```

#include "math.h"

void SETUP(void)
{
    SET_DPS(DPS1, 5.0, V, 100, MA);
    SET_DPS(DPS2, -5.0, V, 100, MA);
    Delay(10);
    SET_PERIOD(488);
    SET_TIMING(100, 350, 450);
    SET_INPUT_LEVEL(2.2, 0.6);
    SET_OUTPUT_LEVEL(2.4, 0.4);
    FORMAT(NRZ0, "48,4,47,46,1,2"); // FSR-1;DR-2;BCLKR-3;PDN-
4;BCLKX-44;MCLKX-45;DX-46; FSX-47; TSX-48
    FORMAT(RZ, "45,44,3");
    RUN_PATTERN(3, 1, 0, 0); // 运行图形 3，产生 FSX 脉冲，使芯片上电
    Delay(10);
}

void PASCAL TP3057()
{
    //ICC1IBB1-TEST
    SETUP();
    SET_DPS(1,5,V,100,MA); //选用 1 号电源通道 DPS1，将 VCC 电压设为 5
    SET_DPS(2,-5,V,100,MA); //选用 2 号电源通道 DPS2，将 VBB 电压设为 5
    RUN_PATTERN(6,1,0,0); //运行 6 号图形文件，使芯片进入上电状态，测量
ICC1IBB1
    Delay(50);

    if(!DPS_MEASURE(DPS1, R20MA, 5, "ICC1",MA, 9, No_LoLimit)) {
        BIN(6);
    }
    if(!DPS_MEASURE(DPS2, R20MA, 5, "IBB1",MA, 9, No_LoLimit)) {
        BIN(6);
    }
}
}

```

9) 参考图形（具体图形见“向量图形.txt”）

索引号 6 号——使芯片进入上电状态，测量 ICC1IBB1： 将 MCLKX 接 2.048 MHz 方波，BCLKX 接 2.048 MHz 方波，MCLKR/PDN 接 GND，FSX 接 8kHz 脉冲，使所有输出端（VFRO、DX、TSX、GSX）悬

空（无负载），VFXI+、VFXI-、FSX、FSR、DR、BCLKR/CLKSEL 接地

5.7. GXA (Transmit Gain, Absolute)

- 1) **参数含义：**从模拟输入端（VF_x 或 GSX）到 PCM 数字输出（DX）的绝对增益。
- 2) **测试类别：**交流参数
- 3) **测试原理：**用标准模拟信号激励编码器，将输出的 PCM 码字按相同压缩律解码重建，测量重建信号的幅度，计算它与输入信号的幅度比（dB），以此验证编码器的绝对增益误差。
- 4) **测试条件：** Input at GSX=0 dBm0 at 1020 Hz
- 5) **被测引脚：**D_x
- 6) **测试步骤**

i) 硬件连接

接 GNDA 到系统地

接 VCC 到 DPS1（+5V）、VBB 到 DPS2（-5V），等待电源稳定

VFXI+接 ST3020 音频电压源（AS）输出，VFXI-短接至 GSX（电阻 R1 开路，R2 = 0Ω），GSX 短接至 VFXI-

DX 接 ST3020 数字通道板（CHB）某输入引脚，MCLKX、BCLKX、FSX 接 ST3020 时钟（MCLK = 1.536/1.544/2.048 MHz，BCLK = 64 kHz ~ 2.048 MHz，FSX = 8 kHz），MCLKR、BCLKR、FSR 可按需接地或悬空

ii) 施加输入信号

生成信号频率：1020Hz，电平：0dBm0（1.2276 V_{rms}）的正弦波输出给 VFXI+

SET_AS(1.2276, V, 1020, HZ)

iii) 读取 DX 输出的 PCM 数据

在每个 BCLKX 下降沿后读取 DX 电平，连续读取 8 位组成一个 PCM 字节。使用 GET_PATTERN_RESULT 与全 0 图形比较（图形文件需有足够多的八位周期以捕捉足够长的 pcm 编码），若读取值为 1 则返回 fail（0x01），若读取值为 0 则返回 pass（0x00），此时 GET_PATTERN_RESULT 输出结果即为 DX 输出的 PCM 编码。

根据图形 7，第一个有效的 FSX 脉冲出现在第 5 个测试周期（从 0 开始计数为 4），紧随其后的 8 个周期（周期 5~12）即为第一个 PCM 字节的 8 个数据位，以此可以确定 PCM 数据的位置，DX 输出的 PCM 编码。

iv) 将 PCM 码字解码为线性电压值

1、还原标准 A-Law 码

TP3057 输出的原始码字是经过“偶数位反转”处理的，需先进行位反转操作，还原成标准的 G.711 A-law 码。

参考代码：unsigned char a_val = pcm_byte ^ 0x55;

2、使用公式将 G.711 A-law 码解码得到 16 位线性 PCM 值

3、计算线性电压值。线性码的范围是 -8031 到 +8031，对应模拟电压的满量程。根据 TP3057 数据手册，A-law 模式的满量程电压是 2.492 V（峰值）

$$V_{\text{peak}} = \text{linear_val} * (2.492 / 8031) \quad \text{Volts}$$

$$V_{\text{rms}} = V_{\text{peak}} / 1.414 \quad \text{Volts}$$

v) 计算增益并判断

增益公式

$$G_{XA} = 20 \log_{10} \left(\frac{V_{\text{out_rms}}}{V_{\text{in_rms}}} \right)$$

其中 $V_{\text{in_rms}} = 1.2276 \text{ Vrms}$ 。

若 $-0.15 \leq G_{XA} \leq 0.15$ ，则测试通过；否则失败

7) 使用的 ATE 资源

AS

8) 参考代码（具体代码见“程序代码.txt”）

```

#include "StdAfx.h"
#include "testdef.h"
#include "data.h"
#include "math.h"

double gxa1020 = 0.0 ;

unsigned char capture_one_pcm_byte(int start_idx, int dx_channel, int
offset);
double pcm_to_voltage_rms(unsigned char tp3057_byte);

void SETUP(void)
{
    SET_DPS(DPS1, 5.0, V, 100, MA);
    SET_DPS(DPS2, -5.0, V, 100, MA);
    Delay(10);
    SET_PERIOD(488);
    SET_TIMING(100, 350, 450);
    SET_INPUT_LEVEL(2.2, 0.6);
    SET_OUTPUT_LEVEL(2.4, 0.4);
    FORMAT(NRZ0, "48,4,47,46,1,2"); // FSR-1;DR-2;BCLKR-3;PDN-
4;BCLKX-44;MCLKX-45;DX-46; FSX-47; TSX-48
    FORMAT(RZ, "45,44,3");
    RUN_PATTERN(3, 1, 0, 0); // 运行图形 3, 产生 FSX 脉冲, 使芯片上电
    Delay(10);
}

void PASCAL TP3057()
{
    //GXA
    SETUP();
    SET_AS(1.2276, V, 1020, HZ); // 音频源输出 1020 Hz, 1.2276
Vrms
    RUN_PATTERN(7, 0, 0, 0);
    unsigned char pcm_byte = capture_one_pcm_byte(7, 46,4); // 图形索
引 7, DX 通道 46,offset = 4;根据图形 7, 第一个 fsx 脉冲将出现在第五个周期 (从
0 开始计数为 4)
    if (pcm_byte == 0xFF) {
        BIN(7); // 捕获失败
        goto AFTER_GXR;//跳过 GXA, GXR
    }
    double vout_rms_gxa = pcm_to_voltage_rms(pcm_byte);
}

```



```

double gain_db = 20.0 * log10(vout_rms_gxa / 1.2276);
gxa1020 = gain_db;
// ===== 判断并显示结果 =====
if (gain_db <= -0.15 || gain_db >= 0.15) {
    //SHOW_RESULT("GXA", gain_db, "dB", 0.15, -0.15);
    BIN(8);
}
}

unsigned char capture_one_pcm_byte(int start_idx, int dx_channel, int
offset)
{
    BYTE* fail_bits = NULL;
    int depth = 0;
    fail_bits = GET_PATTERN_RESULT(start_idx, dx_channel, &depth);
    if (fail_bits == NULL || depth < offset + 9) {
        return 0xFF;
    }
    unsigned char pcm_byte = 0;
    for (int bit = 0; bit < 8; bit++) {
        int cycle = offset + 1 + bit;    // 数据位从 offset+1 开始
        int byte_ofs = cycle / 8;
        int bit_ofs = cycle % 8;
        int level = (fail_bits[byte_ofs] >> bit_ofs) & 1;
        pcm_byte |= (level << (7 - bit));
    }
    return pcm_byte;
}

double pcm_to_voltage_rms(unsigned char tp3057_byte)
{
    // 1. 偶数位反转，恢复标准 A-law
    unsigned char alaw_std = tp3057_byte ^ 0x55;

    // 2. A-law 解码（标准 G.711 算法）
    short linear = 0;
    unsigned char sign = alaw_std & 0x80;
    unsigned char magnitude = alaw_std & 0x7F;
    unsigned char segment = (magnitude >> 4) & 0x07;
    unsigned char step = magnitude & 0x0F;
    if (segment == 0) {

```

```

        linear = (step << 4) + 8;
    } else {
        linear = ((0x10 | step) << (segment + 2)) + 8;
    }
    if (sign) linear = -linear;

    // 3. 转换为峰值电压
    double vpeak = (double)linear * (VPEAK_FS / LINEAR_MAX);
    // 4. 转换为有效值（正弦波）
    return vpeak / sqrt(2.0);
}

```

9) 参考图形（具体图形见附录）

索引号 7 号——读取 PCM 码

5.8. GXR (Transmit Gain, Relative to GXA) (300Hz-3000Hz)

- 1) **参数含义：**以 1020 Hz 时的发送绝对增益（GXA）为参考（0 dB），测量其他频率下发送增益相对于该参考的偏差。
- 2) **测试类别：**交流参数
- 3) **测试原理：**以 1020 Hz、0 dBm 输入下测量的 GXA（绝对增益），作为参考点，保持输入电平不变（0 dBm），改变输入信号频率（300 Hz-3000 Hz）

在每个频率点，捕获 DX 输出的 PCM 码，解码为线性电压，计算该频率下的增益 $G(f)$ ，计算相对增益。

- 4) **测试条件：**输入电平 0dBm（1.2276 Vrms，600Ω），输入频率 300-3000Hz
- 5) **被测引脚：**Dx
- 6) **测试步骤**
 - i) **硬件连接**

与 GXA 相同

ii) 对 300、1000、2000、3000 四个频点单独测量其绝对增益

循环执行：

- a) 设置音频源输出频率 f 、电平 1.2276Vrms
- b) 运行图形，捕捉 8 个周期的失效数据，组装 pcm 字节
- c) 将 pcm 字节解码为 rms 电压 V_{out_rms}
- d) 计算该频率下的绝对增益 G_f
- e) 计算相对增益 $GXR_f = G_f - GXA1020$

iii) 与手册限值比较

判断其是否在 $\pm 0.15\text{dB}$ 之间，否则 bin 分箱

6) 使用的 ATE 资源

AS

8) 参考代码（具体代码见“程序代码.txt”）

```
#include "StdAfx.h"
#include "testdef.h"
#include "data.h"
#include "math.h"

#define VPEAK_FS      2.492          // TP3057 A-law 满量程峰值 (V)
#define LINEAR_MAX    8031          // 线性编码最大值

double gxa1020 = 0.0 ;//gxa1020 具体参数见 GXA-TEST

unsigned char capture_one_pcm_byte(int start_idx, int dx_channel, int offset);
double pcm_to_voltage_rms(unsigned char tp3057_byte);

void SETUP(void)
{
    SET_DPS(DPS1, 5.0, V, 100, MA);
    SET_DPS(DPS2, -5.0, V, 100, MA);
    Delay(10);
    SET_PERIOD(488);
}
```

```

    SET_TIMING(100, 350, 450);
    SET_INPUT_LEVEL(2.2, 0.6);
    SET_OUTPUT_LEVEL(2.4, 0.4);
    FORMAT(NRZ0, "48,4,47,46,1,2"); // FSR-1;DR-2;BCLKR-3;PDN-
4;BCLKX-44;MCLKX-45;DX-46; FSX-47; TSX-48
    FORMAT(RZ, "45,44,3");
    RUN_PATTERN(3, 1, 0, 0); // 运行图形 3, 产生 FSX 脉冲, 使芯片上电
    Delay(10);
}

void PASCAL TP3057()
{
    //GXR
    SETUP();
    double freq_list_gxr[] = {300.0, 1000.0, 2000.0, 3000.0};
    int num_freq_gxr = sizeof(freq_list_gxr) /
sizeof(freq_list_gxr[0]); //计算列表中元素数量
    int fail_count = 0; //gxr 在规定范围之外的个数

    for (int i = 0; i < num_freq_gxr; i++) {
        double f = freq_list_gxr[i];
        SET_AS(1.2276, V, f, HZ);
        Delay(10);

        RUN_PATTERN(7, 0, 0, 0); // 运行图形段 1 (9 个周期)
        unsigned char pcm_byte = capture_one_pcm_byte(7, 46, 4);
        if (pcm_byte == 0xFF) {
            BIN(9); // 捕获失败, 分箱
            fail_count++;
            continue;
        }
        double vout_rms_gxr = pcm_to_voltage_rms(pcm_byte);
        double Gxf = 20.0 * log10(vout_rms_gxr / 1.2276);
        double GXRf = Gxf - gxa1020; //gxa1020 具体参数见 GXA-TEST

        if (GXRf <= -0.15 || GXRf >= 0.15) {
            //SHOW_RESULT("GXR_Hz", GXRf, "dB", 0.15, -0.15);
            fail_count++;
        }
    }

    if (fail_count != 0) BIN(10);
}

```

```

    AFTER_GXR:
}

unsigned char capture_one_pcm_byte(int start_idx, int dx_channel, int
offset)
{
    BYTE* fail_bits = NULL;
    int depth = 0;
    fail_bits = GET_PATTERN_RESULT(start_idx, dx_channel, &depth);
    if (fail_bits == NULL || depth < offset + 9) {
        return 0xFF;
    }
    unsigned char pcm_byte = 0;
    for (int bit = 0; bit < 8; bit++) {
        int cycle = offset + 1 + bit;    // 数据位从 offset+1 开始
        int byte_ofs = cycle / 8;
        int bit_ofs = cycle % 8;
        int level = (fail_bits[byte_ofs] >> bit_ofs) & 1;
        pcm_byte |= (level << (7 - bit));
    }
    return pcm_byte;
}

double pcm_to_voltage_rms(unsigned char tp3057_byte)
{
    // 1. 偶数位反转，恢复标准 A-law
    unsigned char alaw_std = tp3057_byte ^ 0x55;

    // 2. A-law 解码（标准 G.711 算法）
    short linear = 0;
    unsigned char sign = alaw_std & 0x80;
    unsigned char magnitude = alaw_std & 0x7F;
    unsigned char segment = (magnitude >> 4) & 0x07;
    unsigned char step = magnitude & 0x0F;
    if (segment == 0) {
        linear = (step << 4) + 8;
    } else {
        linear = ((0x10 | step) << (segment + 2)) + 8;
    }
    if (sign) linear = -linear;
}

```

```

// 3. 转换为峰值电压
double vpeak = (double)linear * (VPEAK_FS / LINEAR_MAX);
// 4. 转换为有效值（正弦波）
return vpeak / sqrt(2.0);
}

```

9) 参考图形（具体图形见“向量图形.txt”）

索引号 7 号——读取 PCM 码

5.9. GRA (Receive Gain, Absolute)

- 1) 参数含义：从 PCM 数字输入（DR）到模拟音频输出（VFR0）的绝对增益
- 2) 测试类别：交流参数
- 3) 测试原理：用将一份已知幅度、对应 0 dBm 的 A-law PCM 数字码发送到芯片的 DR 输入端，然后测量其模拟输出端 VFR0 的电压幅度，即可评估增益的准确性
- 4) 测试条件： Input = Digital Code Sequence for 0 dBm0 Signal at 1020 Hz
- 5) 被测引脚：VFR0
- 6) 测试步骤

i) 硬件连接

接 GNDA 到系统地

接 VCC 到 DPS1 (+5V)、VBB 到 DPS2 (-5V)，等待电源稳定

DR 接 ST3020 数字输出通道，BCLKR 接收位时钟（2.048 MHz），FSR 接帧同步发生器（8 kHz 脉冲），VFR0 接 ST3020 音频电压表 (AVM) 输入通道，接 600 欧负载至 AGND

未使用引脚：BCLKX/CLKSEL，MCLKR/PDN 接 GND，FSX、BCLKX、MCLKX、VF1+、VF1-、GSX 均悬空，DX 可悬空（三态输出，不使用时建议通过 10kΩ 上拉至 VCC）

ii) 加载并运行图形文件

对 DR 输入包含 1020 Hz、0 dBm0 的 A-law 数字码流（至少一个完整周期）

iii) 测量 VFR0 输出有效值

使用音频电压表（AVM），开启 20 kHz 低通滤波，量程选 2V（或 4V）。测量前等待 10 ms 稳定。

iv) 计算接收绝对增益 GRA 并判断结果

基准电压：0 dBm0 = 1.2276 Vrms，计算公式同 GXA

$\text{double GRA} = 20.0 * \log_{10}(\text{vout_rms} / \text{V_REF_RMS});$

判断 GRA 是否在+0.15dB 范围内

7) 使用的 ATE 资源

DPS、PGCB、CHB、AVM

8) 参考代码（具体代码见“程序代码.txt”）

```
#include "StdAfx.h"
#include "testdef.h"
#include "data.h"
#include "math.h"

#define VPEAK_FS      2.492           // TP3057 A-law 满量程峰值 (V)
#define LINEAR_MAX    8031           // 线性编码最大值

double gra1020 = 0.0 ;//gxa1020 具体参数见 GXA-TEST

unsigned char capture_one_pcm_byte(int start_idx, int dx_channel, int offset);
double pcm_to_voltage_rms(unsigned char tp3057_byte);

void SETUP(void)
{
    SET_DPS(DPS1, 5.0, V, 100, MA);
    SET_DPS(DPS2, -5.0, V, 100, MA);
    Delay(10);
    SET_PERIOD(488);
    SET_TIMING(100, 350, 450);
    SET_INPUT_LEVEL(2.2, 0.6);
    SET_OUTPUT_LEVEL(2.4, 0.4);
```

```

    FORMAT(NRZ0, "48,4,47,46,1,2"); // FSR-1;DR-2;BCLKR-3;PDN-
4;BCLKX-44;MCLKX-45;DX-46; FSX-47; TSX-48
    FORMAT(RZ, "45,44,3");

    RUN_PATTERN(3, 1, 0, 0); // 运行图形 3, 产生 FSX 脉冲, 使芯片上电
    Delay(10);
}

void PASCAL TP3057()
{

    //GRA
    SETUP();
    double vout_rms_gra = 0.0;
    SETUP();
    RUN_PATTERN(10, 0, 0, 0); //10-----1020Hz 0dBm0 PCM 输入
    SET_AVM_PATH(LP20, BPPASS); // 开启 20kHz 低通滤波, 滤除高频噪声
    // 测量 AVM_CHANNEL 通道, 量程 AVM_RANGE_V, 延时 DELAY_MS
    vout_rms_gra = AVM_MEASURE(1, 2.0, V, 10);
    gra1020 = 20.0 * log10(vout_rms_gra / 1.2276);

    // 结果判断及分箱
    if (gra1020 <= -0.15 || gra1020 >= 0.15)
    {
        //SHOW_RESULT("gra1020", gra1020, "dB", 0.15, -0.15);
        BIN(11);
    }

}

```

9) 参考图形（具体图形见附录）

索引号 10 —— 1020Hz 0dBm0 PCM 输入

5.10. GRR (Receive Gain, Relative to GRA) (f = 0Hz to 3000Hz)

- 1) **参数含义：**以 1020 Hz 时的接收绝对增益 (GRA) 为 0 dB 参考，测量其他频率下接收增益相对于该参考的偏差
- 2) **测试类别：**交流参数
- 3) **测试原理：**以 1020 Hz 下的 GRA，作为参考值 (记为 GRA_ref)。保持输入数字码流仍为 0 dBm0，改变码流的频率 (f = 0Hz to 3000Hz)，在每个频率点，测量 VFR0 输出有效值，计算该频率下的绝对增益 G(f)，计算相对增益： $GRR(f) = G(f) - GRA_ref$ 。
- 4) **测试条件：**Input = Digital Code Sequence for 0 dBm0 Signal at 0Hz to 3000Hz
- 5) **被测引脚：**VFR0
- 6) **测试步骤**

i) 硬件连接 (同 GRA)

接 GNDA 到系统地

接 VCC 到 DPS1 (+5V)、VBB 到 DPS2 (-5V)，等待电源稳定

DR 接 ST3020 数字输出通道，BCLKR 接收位时钟 (2.048 MHz)，FSR 接帧同步发生器 (8 kHz 脉冲)，VFR0 接 ST3020 音频电压表 (AVM) 输入通道，接 600 欧负载至 AGND

未使用引脚：BCLKX/CLKSEL，MCLKR/PDN 接 GND，FSX、BCLKX、MCLKX、VF1+、VF1-、GSX 均悬空，DX 可悬空 (三态输出，不使用时建议通过 10kΩ 上拉至 VCC)

ii) 对 300、3000 两个频点单独测量其绝对增益

循环执行：

- a) 加载对应频率的 PCM 码流图形文件
- b) 运行图形
- c) 测量 VFR0 输出有效值
- d) 计算该频率下的绝对增益 Gf

- e) 计算 $GRR(f) = G(f) - GRA_{ref}$
- f) 与限制比较, 判断是否在 $\pm 0.15\text{dB}$ 内

7) 使用的 ATE 资源

DPS、PGCB、CHB、AVM

8) 参考代码 (具体代码见“程序代码.txt”)

```
#include "StdAfx.h"
#include "testdef.h"
#include "data.h"
#include "math.h"

double gra1020 = 0.0 ; //gra1020 由 gra-test 测得

void SETUP(void)
{
    SET_DPS(DPS1, 5.0, V, 100, MA);
    SET_DPS(DPS2, -5.0, V, 100, MA);
    Delay(10);
    SET_PERIOD(488);
    SET_TIMING(100, 350, 450);
    SET_INPUT_LEVEL(2.2, 0.6);
    SET_OUTPUT_LEVEL(2.4, 0.4);
    FORMAT(NRZ0, "48,4,47,46,1,2"); // FSR-1;DR-2;BCLKR-3;PDN-
4;BCLKX-44;MCLKX-45;DX-46; FSX-47; TSX-48
    FORMAT(RZ, "45,44,3");

    RUN_PATTERN(3, 1, 0, 0); // 运行图形 3, 产生 FSX 脉冲, 使芯片上电
    Delay(10);
}

void PASCAL TP3057()
{
    //GRR
    SETUP();
    int start_idx_list[] = {8, 12}; // 分别对应 300,3000 Hz 0dBm0 PCM
    输入
    double freq_list_grr[] = {300.0, 3000.0};
```

```

    int num_freq_grr = sizeof(freq_list_grr) /
sizeof(freq_list_grr[0]); //计算列表中元素数量
    int fail = 0;

    // 循环测试
    for (int i = 0; i < num_freq_grr; i++)
    {
        // 运行对应频率的图形
        RUN_PATTERN(start_idx_list[i], 0, 0, 0);
        SET_AVM_PATH(LP20, BPPASS);
        double vout_rms_grr = AVM_MEASURE(1, 2.0, V, 10);
        double Grf = 20.0 * log10(vout_rms_grr / 1.2276);
        double GRR = Grf - gra1020; //gra1020 由 gra-test 测得

        if (GRR <= -0.15 || GRR >= 0.15)
        {
            fail++;
        }
    }
    if (fail != 0) BIN(12); //分箱

    AFTER_GRR:
}

```

9) 参考图形（具体图形见“向量图形.txt”）

索引号 8 —— 300Hz 0dBm0 PCM 输入

索引号 12 —— 3000Hz 0dBm0 PCM 输入

5.11. GRRL (Receive Gain Variations with Level) (-40 dBm0 to +3 dBm0)

1) 参数含义：GRRL 表示接收通路的增益线性度，即在不同输入电平下，接

收增益相对于参考电平（通常为 -10 dBm0）的偏差

- 2) **测试类别：**交流参数
- 3) **测试原理：**固定频率 1020 Hz，改变输入 PCM 码流的幅度（对应不同的 dBm0 值）。以 0 dBm0 作为参考电平，测量 VFR0 输出有效值，计算该点的绝对增益 G_{ref} 。对于每个测试电平（-40、+3 dBm0），测量输出有效值，计算绝对增益 $G(L)$ ，计算增益变化 $\Delta G(L) = G(L) - G_{ref}$ ，判断 $\Delta G(L)$ 是否在 ± 0.2 dB 内
- 4) **测试条件：** $f = 1020\text{Hz}$
- 5) **被测引脚：**VFR0
- 6) **测试步骤**
 - i) **硬件连接（同 GRA）**

接 GNDA 到系统地

接 VCC 到 DPS1（+5V）、VBB 到 DPS2（-5V），等待电源稳定

DR 接 ST3020 数字输出通道，BCLKR 接收位时钟（2.048 MHz），FSR 接帧同步发生器（8 kHz 脉冲），VFR0 接 ST3020 音频电压表（AVM）输入通道 1，并接 600 欧负载至 AGND

未使用引脚：BCLKX/CLKSEL，MCLKR/PDN 接 GND，FSX、BCLKX、MCLKX、VF1+、VF1-、GSX 均悬空，DX 可悬空（三态输出，不使用时建议通过 10k Ω 上拉至 VCC）
 - ii) **测量参考电平 0dBm0 的增益**

运行 1020Hz，0dBm0 的图形文件，测量 VFR0 输出有效值，计算绝对增益 G_{ref}
 - iii) **对-40、+3 dBm0 两个电平单独测量其绝对增益，并与参考增益比较**

循环执行：

 - a) 加载对应电平的 PCM 码流图形文件
 - b) 运行图形
 - c) 测量 VFR0 输出有效值
 - d) 计算该频率下的绝对增益 G_{abs}

- e) 计算 $GRRL = G_{abs} - G_{ref}$
- f) 与限制比较, 判断是否在 $\pm 0.2\text{dB}$ 内

7) 使用的 ATE 资源

DPS、PGCB、CHB、AVM

8) 参考代码 (具体代码见“程序代码.txt”)

```
#include "StdAfx.h"
#include "testdef.h"
#include "data.h"
#include "math.h"

double gra1020 = 0.0 ;//由 gra-test 测得

void SETUP(void)
{
    SET_DPS(DPS1, 5.0, V, 100, MA);
    SET_DPS(DPS2, -5.0, V, 100, MA);
    Delay(10);
    SET_PERIOD(488);
    SET_TIMING(100, 350, 450);
    SET_INPUT_LEVEL(2.2, 0.6);
    SET_OUTPUT_LEVEL(2.4, 0.4);
    FORMAT(NRZ0, "48,4,47,46,1,2"); // FSR-1;DR-2;BCLKR-3;PDN-
4;BCLKX-44;MCLKX-45;DX-46; FSX-47; TSX-48
    FORMAT(RZ, "45,44,3");

    RUN_PATTERN(3, 1, 0, 0); // 运行图形 3, 产生 FSX 脉冲, 使芯片上电
    Delay(10);
}

void PASCAL TP3057()
{
    //GRRL
    SETUP();
    // 电平列表 (dBm0)
    double level_dbm0[] = {-40.0, 3.0};
    // 对应的 START_INDEX 序号
    int index[] = {9, 11};
```

```

int num = 2; //选测 2 个点

RUN_PATTERN(10, 0, 0, 0);           // 运行 0 dBm0 码流段
SET_AVM_PATH(LP20, BPPASS);         // 20kHz 低通滤波
double Vref = AVM_MEASURE(1, 4.0, V, 10);
// 参考绝对增益（可不计算，直接记录 Vref 用于相对比较）
double Gref = 20.0 * log10(Vref / 1.2276); // 1.2276 是 0 dBm0 理想电压

int fail_grrl = 0;
for (int i = 0; i < num; i++)
{
    int idx = index[i];
    double L = level_dbm0[i];
    RUN_PATTERN(idx, 0, 0, 0);
    double vout_grrl = AVM_MEASURE(1, 4.0, V, 10);
    double Gabs = 20.0 * log10(vout_grrl / 1.2276);
    double delta = Gabs - Gref;           // 增益变化

    // 判断限值：若电平在 -40 ~ +3 dBm0 范围内，允许 ±0.2 dB
    if (delta < -0.2 || delta > 0.2) {
        fail_grrl++;
    }
}
if (fail_grrl != 0) BIN(13);

AFTER_GRR:
}

```

9) 参考图形（具体图形见附录）

索引号 9 —— 1020Hz -40dBm0 PCM 输入

索引号 11 —— 1020Hz 3dBm0 PCM 输入

5.12. SFDX (Single Frequency Distortion, Transmit)

- 1) **参数含义:** 编码器在纯正弦波输入下, 其输出 PCM 解码后信号中谐波分量相对于基波的幅度
- 2) **测试类别:** 交流参数
- 3) **测试原理:** 向 TP3057 的模拟输入端 (VF1+) 送入一个 1020 Hz, 0 dBm0 正弦波, 读取 DX 脚的 PCM 码输出, 转化为线性电压序列, 并对该序列进行 FFT 分析, 得到频谱, 找出基波 (1020 Hz) 的幅度和最大谐波 (如 2040 Hz、3060 Hz) 的幅度, 计算谐波与基波的幅度比 (dB), 即单频失真 SFDX。
- 4) **测试条件:** VF1+输入 1020 Hz, 0 dBm0 正弦波
- 5) **被测引脚:** DX
- 6) **测试步骤**
 - i) **硬件连接 (同 GXA)**

接 GNDA 到系统地

接 VCC 到 DPS1 (+5V)、VBB 到 DPS2 (-5V), 等待电源稳定

VFXI+接 ST3020 音频电压源 (AS) 输出, VFXI-短接至 GSX (电阻 R1 开路, $R2 = 0\Omega$), GSX 短接至 VFXI-

DX 接 ST3020 数字通道板 (CHB) 某输入引脚, MCLKX、BCLKX、FSX 接 ST3020 时钟 (MCLK = 1.536/1.544/2.048 MHz, BCLK = 64 kHz ~ 2.048 MHz, FSX = 8 kHz), MCLKR、BCLKR、FSR 可按需接地或悬空
 - ii) **音频源输出**

输出 1020Hz, 1.2276Vrms 的正弦波
 - iii) **运行图形 (预期 DX 全 0)**

捕捉足够长的 PCM 码流 (例如 **4096 个周期**, 即 4096 个 PCM 位, 对应 512 个字节)。图形文件中使用 RPT 重复 INC (L) 指令
 - iv) **获取失效数据**

使用 GET_PATTERN_RESULT 读取每个周期的 DX 电平 (0/1), 按 8

位一组还原 PCM 字节。

v) 解码成线性电压

对每个 PCM 字节进行偶数位反转恢复标准 A-law，再解码为线性值（范围 -8031~8031）。得到一个电压序列

vi) 应用窗函数（如汉宁窗）减少频谱泄漏

vii) 执行 FFT，计算幅度谱

viii) 找出基波（1020 Hz）幅度和最大谐波幅度（通常为二次谐波 2040 Hz）

ix) 计算单频失真： $SFDX = 20 \log_{10}(V_{spr}/V_f)$

x) 与 -46 dB 比较

7) 使用的 ATE 资源

DPS、AS、AVM

8) 参考代码（具体代码见“程序代码.txt”）

```
//TP3057???Q???  
  
#include "StdAfx.h"  
#include "testdef.h"  
#include "data.h"  
#include "math.h"  
  
//sfdx  
const double M_PI = 3.14159265358979323846;  
#define SFDX_FFT_POINTS 512 // FFT 点数（同时是捕获的 PCM 字节数）  
#define SFDX_SAMPLE_RATE 8000.0 // PCM 采样率 (Hz)  
  
double pcm_to_voltage_rms(unsigned char tp3057_byte);  
int capture_multi_pcm_bytes(int start_idx, int dx_channel, int offset,  
                             unsigned char* pcm_bytes, int num_bytes);  
void apply_hanning(double* data, int n);  
  
void SETUP(void)  
{
```



```

    SET_DPS(DPS1, 5.0, V, 100, MA);
    SET_DPS(DPS2, -5.0, V, 100, MA);
    Delay(10);
    SET_PERIOD(488);
    SET_TIMING(100, 350, 450);
    SET_INPUT_LEVEL(2.2, 0.6);
    SET_OUTPUT_LEVEL(2.4, 0.4);
    FORMAT(NRZ0, "48,4,47,46,1,2"); // FSR-1;DR-2;BCLKR-3;PDN-
4;BCLKX-44;MCLKX-45;DX-46; FSX-47; TSX-48
    FORMAT(RZ, "45,44,3");

    RUN_PATTERN(3, 1, 0, 0); // 运行图形 3, 产生 FSX 脉冲, 使芯片上电
    Delay(10);
}

void PASCAL TP3057()
{

    // ===== SFDX 测试 (发送单频失真) =====

    // 1. 初始化环境 (确保芯片上电、时钟等)
    SETUP(); // 设置电源、时钟、FORMAT
    SET_AS(1.2276, V, 1020, HZ); // 输入 1020 Hz, 0 dBm0
    Delay(20); // 等待稳定

    // 2. 运行图形 7 (预期 DX 全 L, 失效继续)
    RUN_PATTERN(7, 0, 0, 0);

    // 3. 捕获多个 PCM 字节
    unsigned char pcm_data_sfdx[SFDX_FFT_POINTS];
    int captured_sfdx = capture_multi_pcm_bytes(7, 46, 4,
pcm_data_sfdx, SFDX_FFT_POINTS);
    if (captured_sfdx != SFDX_FFT_POINTS) {
        BIN(14); // 捕获失败分箱
        goto END_SFDX;
    }

    // 4. 解码为电压序列 (有效值, 但 FFT 需要瞬时值, 这里直接使用峰值电压)
    double voltage_sfdx[SFDX_FFT_POINTS];
    for (int i = 0; i < SFDX_FFT_POINTS; i++) {
        double vrms = pcm_to_voltage_rms(pcm_data_sfdx[i]);
        // 为了 FFT, 转换为峰值电压 (正弦波峰值 = RMS * sqrt(2))
        voltage_sfdx[i] = vrms * sqrt(2.0);
    }
}

```

```

}

// 5. 加窗
apply_hanning(voltage_sfdx, SFDX_FFT_POINTS);

// 6. FFT
double fft_mag_sfdx[SFDX_FFT_POINTS / 2];
if (!DFT(voltage_sfdx, SFDX_FFT_POINTS, fft_mag_sfdx)) {
    BIN(15);    // FFT 失败
    goto END_SFDX;
}

// 7. 计算频率分辨率, 找到基波和谐波 bin
double freq_res_sfdx = SFDX_SAMPLE_RATE / SFDX_FFT_POINTS;
int bin_fund_sfdx = (int)(1020.0 / freq_res_sfdx + 0.5);
int bin_2nd_sfdx = (int)(2 * 1020.0 / freq_res_sfdx + 0.5);
int bin_3rd_sfdx = (int)(3 * 1020.0 / freq_res_sfdx + 0.5);

if (bin_2nd_sfdx >= SFDX_FFT_POINTS/2 || bin_3rd_sfdx >=
SFDX_FFT_POINTS/2) {
    BIN(16);    // 频率超出范围
    goto END_SFDX;
}

double A1_sfdx = fft_mag_sfdx[bin_fund_sfdx];
double A2_sfdx = fft_mag_sfdx[bin_2nd_sfdx];
double A3_sfdx = fft_mag_sfdx[bin_3rd_sfdx];
double Ah_sfdx = (A2_sfdx > A3_sfdx) ? A2_sfdx : A3_sfdx;
double SFDX_db = 20.0 * log10(Ah_sfdx / A1_sfdx);

// 8. 判断是否合格 (MAX -46 dB)
if (!(SFDX_db > -46)) {
    SHOW_RESULT("SFDX", SFDX_db, "dB", -46, No_LoLimit);
    BIN(17);    // 失真超限
}
else
{
    SHOW_RESULT("SFDX", SFDX_db, "dB", -46, No_LoLimit);
}

END_SFDX:

```

```

}

double pcm_to_voltage_rms(unsigned char tp3057_byte)
{
    // 1. 偶数位反转, 恢复标准 A-law
    unsigned char alaw_std = tp3057_byte ^ 0x55;

    // 2. A-law 解码 (标准 G.711 算法)
    short linear = 0;
    unsigned char sign = alaw_std & 0x80;
    unsigned char magnitude = alaw_std & 0x7F;
    unsigned char segment = (magnitude >> 4) & 0x07;
    unsigned char step = magnitude & 0x0F;
    if (segment == 0) {
        linear = (step << 4) + 8;
    } else {
        linear = ((0x10 | step) << (segment + 2)) + 8;
    }
    if (sign) linear = -linear;

    // 3. 转换为峰值电压
    double vpeak = (double)linear * (VPEAK_FS / LINEAR_MAX);
    // 4. 转换为有效值 (正弦波)
    return vpeak / sqrt(2.0);
}

// 捕获多个 PCM 字节 (从图形 start_idx 中, 从 offset 周期开始提取)
// 返回实际捕获的字节数, 数据存入 pcm_bytes 数组 (至少 num_bytes 长度)
int capture_multi_pcm_bytes(int start_idx, int dx_channel, int
offset,
                           unsigned char* pcm_bytes, int num_bytes)
{
    BYTE* fail_bits = NULL;
    int depth = 0;
    fail_bits = GET_PATTERN_RESULT(start_idx, dx_channel, &depth);
    if (fail_bits == NULL || depth < offset + num_bytes * 9) {
        return 0;    // 数据不足
    }
}

```

```

    for (int byte = 0; byte < num_bytes; byte++) {
        unsigned char pcm_byte = 0;
        for (int bit = 0; bit < 8; bit++) {
            int cycle = offset + 1 + byte*256 + bit;    // 第五字节为 fsx
置 1, offset 为 4, 每个字节占用 256 个周期, fsx-8khz
            int byte_ofs = cycle / 8;
            int bit_ofs = cycle % 8;
            int level = (fail_bits[byte_ofs] >> bit_ofs) & 1;
            pcm_byte |= (level << (7 - bit));
        }
        pcm_bytes[byte] = pcm_byte;
    }
    return num_bytes;
}

void apply_hanning(double* data, int n) //加汉宁窗
{
    for (int i = 0; i < n; i++) {
        double w = 0.5 * (1 - cos(2 * M_PI * i / (n - 1)));
        data[i] *= w;
    }
}

```

9) 参考图形（具体图形见“向量图形.txt”）

索引号 7 号——读取 PCM 码

5.13. SFDR (Single Frequency Distortion, Receive)

- 1) 参数含义：在接收端数字输入引脚 D_x 上施加一个标准正弦波（0 dBm₀）对应的 PCM 码流，然后测量接收模拟输出 VFR0 的单频失真，要求 ≤ -46 dB
- 2) 测试类别：交流参数
- 3) 测试原理：向 DR 引脚输入一个 **1020 Hz、0 dBm₀** 的 A-law PCM 数字码

流（已包含偶数位反转）。TP3057 内部 DAC 将数字码转换为模拟信号，从 VFR0 输出。用 **高速采样（DVM）** 捕获 VFR0 的时域波形。对波形进行 **FFT 分析**，得到基波（1020 Hz）和各次谐波幅度。计算单频失真：

$$SFD_R = 20\log_{10}\left(\frac{\max(A_2, A_3, \dots)}{A_1}\right) \text{ dB}$$

典型值为 -46 dB，即谐波比基波低 46 dB。

4) **测试条件：**DR 输入 1020 Hz，0 dBm0 A-law PCM 数字码流

5) **被测引脚：**VFR0

6) **测试步骤**

i) **硬件连接**

接 GNDA 到系统地

接 VCC 到 DPS1（+5V）、VBB 到 DPS2（-5V），等待电源稳定

DR 接 CHB 数字通道输出，VFR0 接 DVM 通道 1，并接 600 欧负载到

GNDA

MCLKX、BCLKX、FSR 接 ST3020 时钟（MCLK = 2.048 MHz，BCLK = 2.048 MHz，FSX = 8 kHz），DX，MCLKR、BCLKR、FSR 可按需接地或悬空

ii) **加载运行图形**

发送 1020Hz，0 dBm0 的 PCM 码流，256 字节

iii) **用 DVM 连续采样 VFR0 输出**

捕捉足够长的 PCM 码流（256 字节）。采样点数 ipNumber=2048，采样频率 100kHz，分配系数 freq=500

iv) **应用窗函数**（如汉宁窗）减少频谱泄漏

v) **执行 FFT**，计算幅度谱

vi) **找出基波（1020 Hz）幅度和最大谐波幅度**（通常为二次谐波 2040 Hz）

vii) **计算单频失真：** $SFDR=20\log_{10}(V_{spr}/V_f)$

viii) **与 -46 dB 比较**

7) **使用的 ATE 资源**

DPS、DVM

8) 参考代码（具体代码见“程序代码.txt”）

```
#include "StdAfx.h"
#include "testdef.h"
#include "data.h"
#include "math.h"

const double M_PI = 3.14159265358979323846;
// SFDR 测试参数
#define SFDR_FFT_POINTS    1024        // FFT 点数（必须是 2 的幂）
#define SFDR_DVM_SR        100000.0    // DVM 采样率 (Hz), 100 kHz
#define SFDR_FREQ_RES      (SFDR_DVM_SR / SFDR_FFT_POINTS) // 频率分辨率

double pcm_to_voltage_rms(unsigned char tp3057_byte);
int capture_multi_pcm_bytes(int start_idx, int dx_channel, int
offset,
                           unsigned char* pcm_bytes, int num_bytes);
void apply_hanning(double* data, int n);

void SETUP(void)
{
    SET_DPS(DPS1, 5.0, V, 100, MA);
    SET_DPS(DPS2, -5.0, V, 100, MA);
    Delay(10);
    SET_PERIOD(488);
    SET_TIMING(100, 350, 450);
    SET_INPUT_LEVEL(2.2, 0.6);
    SET_OUTPUT_LEVEL(2.4, 0.4);
    FORMAT(NRZ0, "48,4,47,46,1,2"); // FSR-1;DR-2;BCLKR-3;PDN-
4;BCLKX-44;MCLKX-45;DX-46; FSX-47; TSX-48
    FORMAT(RZ, "45,44,3");

    RUN_PATTERN(3, 1, 0, 0); // 运行图形 3, 产生 FSX 脉冲, 使芯片上电
    Delay(10);
}

void PASCAL TP3057()
```

```

{
    // ===== SFDR_R 测试（接收单频失真） =====

    // 1. 初始化环境（确保芯片上电，接收时钟正常）
    SETUP(); // 设置电源、时钟、FORMAT

    // 2. 运行接收图形 10（1020Hz 0dBm0 PCM 码流）
    if (!RUN_PATTERN(10, 0, 0, 0)) {
        BIN(18); // 图形运行失败
        goto END_SFDR;
    }
    Delay(10); // 等待信号稳定

    // 5. 用 DVM 连续采样 VFR0（通道 1）
    double voltage_sfdr[SFDR_FFT_POINTS];
    // 采样率 = 50MHz / freq_div, 计算分频系数
    int freq_div = (int)(50e6 / SFDR_DVM_SR);
    // 参数: 通道 1, 量程 2V, 延迟 10ms, 采样点数, 分频系数, 数据数组
    MAT_DVM_MEASURE(1, 2.0, V, 10, SFDR_FFT_POINTS, freq_div,
voltage_sfdr);

    // 4. 加汉宁窗
    for (int i = 0; i < SFDR_FFT_POINTS; i++) {
        double w = 0.5 * (1 - cos(2 * M_PI * i / (SFDR_FFT_POINTS -
1)));
        voltage_sfdr[i] *= w;
    }

    // 5. FFT 变换
    double fft_mag_sfdr[SFDR_FFT_POINTS / 2];
    if (!DFT(voltage_sfdr, SFDR_FFT_POINTS, fft_mag_sfdr)) {
        BIN(19); // FFT 失败
        goto END_SFDR;
    }

    // 6. 查找基波和谐波幅度
    int bin_fund_sfdr = (int)(1020 / SFDR_FREQ_RES + 0.5);
    int bin_2nd_sfdr = (int)(2 * 1020 / SFDR_FREQ_RES + 0.5);
    int bin_3rd_sfdr = (int)(3 * 1020 / SFDR_FREQ_RES + 0.5);

    if (bin_2nd_sfdr >= SFDR_FFT_POINTS/2 || bin_3rd_sfdr >=
SFDR_FFT_POINTS/2) {

```

```

        BIN(20);    // 谐波超出奈奎斯特频率
        goto END_SFDR;
    }

    double A1_sfdr = fft_mag_sfdr[bin_fund_sfdr];
    double A2_sfdr = fft_mag_sfdr[bin_2nd_sfdr];
    double A3_sfdr = fft_mag_sfdr[bin_3rd_sfdr];
    double Ah_sfdr = (A2_sfdr > A3_sfdr) ? A2_sfdr : A3_sfdr;
    double SFDR_db = 20.0 * log10(Ah_sfdr / A1_sfdr);

    // 7. 判断合格性 (要求 SFDR_db 优于 -46dB, >-46 为不合格)
    if (SFDR_db > -46)
    {
        SHOW_RESULT("SFDR", SFDR_db, "dB", -46, No_LoLimit);
        BIN(21);    // 失真超限
    }
    else
    {
        SHOW_RESULT("SFDR", SFDR_db, "dB", -46, No_LoLimit);
    }

    END_SFDR:
}

double pcm_to_voltage_rms(unsigned char tp3057_byte)
{
    // 1. 偶数位反转, 恢复标准 A-law
    unsigned char alaw_std = tp3057_byte ^ 0x55;

    // 2. A-law 解码 (标准 G.711 算法)
    short linear = 0;
    unsigned char sign = alaw_std & 0x80;
    unsigned char magnitude = alaw_std & 0x7F;
    unsigned char segment = (magnitude >> 4) & 0x07;
    unsigned char step = magnitude & 0x0F;
    if (segment == 0) {
        linear = (step << 4) + 8;
    } else {
        linear = ((0x10 | step) << (segment + 2)) + 8;
    }
}

```



```

    if (sign) linear = -linear;

    // 3. 转换为峰值电压
    double vpeak = (double)linear * (VPEAK_FS / LINEAR_MAX);
    // 4. 转换为有效值（正弦波）
    return vpeak / sqrt(2.0);
}

// 捕获多个 PCM 字节（从图形 start_idx 中，从 offset 周期开始提取）
// 返回实际捕获的字节数，数据存入 pcm_bytes 数组（至少 num_bytes 长度）
int capture_multi_pcm_bytes(int start_idx, int dx_channel, int
offset,
                           unsigned char* pcm_bytes, int num_bytes)
{
    BYTE* fail_bits = NULL;
    int depth = 0;
    fail_bits = GET_PATTERN_RESULT(start_idx, dx_channel, &depth);
    if (fail_bits == NULL || depth < offset + num_bytes * 9) {
        return 0;    // 数据不足
    }
    for (int byte = 0; byte < num_bytes; byte++) {
        unsigned char pcm_byte = 0;
        for (int bit = 0; bit < 8; bit++) {
            int cycle = offset + 1 + byte*256 + bit;    // 第五字节为 fsx
置 1, offset 为 4, 每个字节占用 256 个周期, fsx=8khz
            int byte_ofs = cycle / 8;
            int bit_ofs = cycle % 8;
            int level = (fail_bits[byte_ofs] >> bit_ofs) & 1;
            pcm_byte |= (level << (7 - bit));
        }
        pcm_bytes[byte] = pcm_byte;
    }
    return num_bytes;
}

void apply_hanning(double* data, int n) //加汉宁窗
{
    for (int i = 0; i < n; i++) {
        double w = 0.5 * (1 - cos(2 * M_PI * i / (n - 1)));
        data[i] *= w;
    }
}

```

}
}

9) 参考图形（具体图形见附录）

索引号 10 号——1020Hz 0dBm0 PCM 输入

5.14. IDM (Intermodulation Distortion)

- 1) **参数含义：**向模拟输入端（VF1+）注入两个频率（例如 300 Hz 和 3400 Hz）的混合信号，电平范围 -4 dBm0 ~ -21 dBm0（通常取中间值如 -10 dBm0），通过分析 DX 输出的 PCM 解码后频谱，计算互调产物的幅度相对于基波的比值。
- 2) **测试类别：**交流参数
- 3) **测试原理：**生成双音信号：频率 $f_1 = 300$ Hz 和 $f_2 = 3400$ Hz，幅度相等，总电平为 -10 dBm0（每个单音电平 -13 dBm0）。通过音频源（AS）输出该双音信号到 VF1+。捕获 DX 输出的 PCM 码流，解码为线性电压序列。对序列进行 FFT，得到频谱。找出两个基波幅度 A_{f1} 和 A_{f2} ，以及主要的互调产物（二阶： $f_1 \pm f_2$ ，三阶： $2f_1 \pm f_2$ 、 $2f_2 \pm f_1$ 等，重点关注落在音频带内的成分，例如 $f_2 - f_1$ 、 $f_1 + f_2$ 等）。取最大互调产物的幅度 A_{im} ，计算 IMD：

$$\text{IMD} = 20 \log_{10} \left(\frac{A_{im}}{\max(A_{f1}, A_{f2})} \right) \text{ dB}$$

要求 $\text{IMD} \leq -41$ dB。

- 4) **测试条件：**VF1+输入 300Hz 和 3400Hz，总电平-10 dBm0 正弦波
- 5) **被测引脚：**DX
- 6) **测试步骤**
 - i) **硬件连接（同 GXA/SFDX）**

接 GNDA 到系统地

接 VCC 到 DPS1 (+5V)、VBB 到 DPS2 (-5V)，等待电源稳定

VFXI+接 ST3020 音频电压源 (AS) 输出，VFXI-短接至 GSX (电阻 R1 开路， $R2 = 0\Omega$)，GSX 短接至 VFXI-

DX 接 ST3020 数字通道板 (CHB) 某输入引脚，MCLKX、BCLKX、FSX 接 ST3020 时钟 ($MCLK = 2.048\text{ MHz}$ ， $BCLK = 2.048\text{ MHz}$ ， $FSX = 8\text{ kHz}$)，MCLKR、BCLKR、FSR 可按需接地或悬空

ii) 音频源输出双音信号

ST3020 的 SET_AS 只能输出单音，因此需要自定义波形：

用 LOAD_AS_PATTERN 载入一个双音波形的表，然后

用 RUN_AS_PATTERN 输出。双音波形生成：采样率 100 kHz ，点数 1024，频率 $f_1 = 300\text{ Hz}$ 和 $f_2 = 3400\text{ Hz}$ 。

每个单音峰值电压 $V_{\text{peak}} = 1.2276 \times 10^{-10/20} \approx 0.388\text{ V}$ (总有效值 $0.388\text{ V}_{\text{rms}}$)。

合成波形： $v(t) = V_{\text{peak}}[\sin(2\pi f_1 t) + \sin(2\pi f_2 t)]$ ，映射到 $0 \sim 65535$ (对应 $\pm 10\text{ V}$ 满量程)。

iii) 运行图形 (预期 DX 全 0)

捕捉足够长的 PCM 码流 (4096 个周期，即 4096 个 PCM 位，对应 512 个字节)。

iv) 获取失效数据

使用 GET_PATTERN_RESULT 读取每个周期的 DX 电平 (0/1)，按 8 位一组还原 PCM 字节。

v) 解码成线性电压

对每个 PCM 字节进行偶数位反转恢复标准 A-law，再解码为线性值 (范围 $-8031 \sim 8031$)。得到一个电压序列

vi) 应用窗函数 (如汉宁窗) 减少频谱泄漏

vii) 执行 FFT，计算幅度谱

viii) 识别互调产物：在频谱中寻找 $f_2 - f_1$ 、 $f_1 + f_2$ 等频率点 (例如 3100

Hz 和 3700 Hz)

ix) 计算 **IMD**: 取最大互调幅度与最大基波幅度之比, 转换为 dB

x) 与 **-41 dB** 比较: 要求 $IDM \leq 41dB$

7) 使用的 ATE 资源

DPS, PGCB, CHB, AS, DFT

8) 参考代码 (具体代码见“程序代码.txt”)

```
//TP3057???Q???

#include "StdAfx.h"
#include "testdef.h"
#include "data.h"
#include "math.h"

const double M_PI = 3.14159265358979323846;
// 双音参数
// ----- 本地常量 -----
const double IMD_F1 = 300.0;
const double IMD_F2 = 3400.0;
const double IMD_TEST_LEVEL_DBM = -10.0;
const double IMD_V_TOTAL_RMS = 1.2276 * pow(10.0, IMD_TEST_LEVEL_DBM
/ 20.0); // 0.388 Vrms
const double IMD_V_SINGLE_PEAK = IMD_V_TOTAL_RMS; // 每个单音峰值
const int IMD_PCM_BYTES = 512;
const int IMD_FFT_POINTS = IMD_PCM_BYTES;
const double IMD_FS_PCM = 8000.0;
const int IMD_AS_POINTS = 1024;
const double IMD_AS_SAMPLE_RATE = 100000.0;
const int IMD_AS_FREQ_DIV = (int)(50e6 / IMD_AS_SAMPLE_RATE); // 500
// 图形 7 专用参数
const int IMD_FIRST_FSX = 4; // 第一个 FSX 脉冲周期索引

double pcm_to_voltage_rms(unsigned char tp3057_byte);
int capture_multi_pcm_bytes(int start_idx, int dx_channel, int
offset,
                                unsigned char* pcm_bytes, int num_bytes);
```

```

void apply_hanning(double* data, int n);

void SETUP(void)
{
    SET_DPS(DPS1, 5.0, V, 100, MA);
    SET_DPS(DPS2, -5.0, V, 100, MA);
    Delay(10);
    SET_PERIOD(488);
    SET_TIMING(100, 350, 450);
    SET_INPUT_LEVEL(2.2, 0.6);
    SET_OUTPUT_LEVEL(2.4, 0.4);
    FORMAT(NRZ0, "48,4,47,46,1,2"); // FSR-1;DR-2;BCLKR-3;PDN-
4;BCLKX-44;MCLKX-45;DX-46; FSX-47; TSX-48
    FORMAT(RZ, "45,44,3");
    RUN_PATTERN(3, 1, 0, 0); // 运行图形 3, 产生 FSX 脉冲, 使芯片上电
    Delay(10);
}

void PASCAL TP3057()
{
    //IDM
    // ----- 生成并播放双音波形 -----
    WORD as_waveform[IMD_AS_POINTS];
    for (int i = 0; i < IMD_AS_POINTS; i++) {
        double t = i / IMD_AS_SAMPLE_RATE;
        double v = IMD_V_SINGLE_PEAK * (sin(2 * M_PI * IMD_F1 * t) +
sin(2 * M_PI * IMD_F2 * t));
        if (v > 10) v = 10;
        if (v < -10) v = -10;
        as_waveform[i] = (WORD)(((v / 10) + 1.0) / 2.0 * 65535.0 +
0.5);
    }
    LOAD_AS_PATTERN(7, 1, as_waveform);
    RUN_AS_PATTERN(7, 1, IMD_AS_FREQ_DIV, 10);

    // -----运行图形 7 (预期 DX 全 L, 失效继续) -----
    RUN_PATTERN(7, 0, 0, 0);

    // ----- 从失效存储器捕获 PCM 字节 -----

    unsigned char pcm_data_idm[IMD_PCM_BYTES];

```

```

    int captured_idm = capture_multi_pcm_bytes(7, 46, 4, pcm_data_idm,
IMD_PCM_BYTES);
    if (captured_idm != IMD_PCM_BYTES) {
        BIN(22);    // 捕获失败分箱
        goto END_SFDX;
    }

    // ----- 6. 解码为电压序列（使用现有 pcm_to_voltage_rms 并转成峰
值） -----
    double voltage_imd[IMD_PCM_BYTES];
    for (int i = 0; i < IMD_PCM_BYTES; i++) {
        double vrms = pcm_to_voltage_rms(pcm_data_idm[i]);
        voltage_imd[i] = vrms * sqrt(2.0);    // 转换为峰值电压
    }

    // ----- 7. 加窗（调用现有 apply_hanning） -----
    apply_hanning(voltage_imd, IMD_PCM_BYTES);

    // ----- 8. FFT -----
    double fft_mag_idm[IMD_FFT_POINTS / 2];
    if (!DFT(voltage_imd, IMD_FFT_POINTS, fft_mag_idm)) {
        BIN(23);
        goto END_IMD;
    }

    // ----- 9. 查找基波和互调产物 -----
    double freq_res_idm = IMD_FS_PCM / IMD_FFT_POINTS;
    int bin_f1 = (int)(IMD_F1 / freq_res_idm + 0.5);
    int bin_f2 = (int)(IMD_F2 / freq_res_idm + 0.5);
    int bin_sum = (int)((IMD_F1 + IMD_F2) / freq_res_idm + 0.5);
    int bin_diff = (int)((IMD_F2 - IMD_F1) / freq_res_idm + 0.5);

    if (bin_f1 >= IMD_FFT_POINTS/2 || bin_f2 >= IMD_FFT_POINTS/2 ||
        bin_sum >= IMD_FFT_POINTS/2 || bin_diff >= IMD_FFT_POINTS/2) {
        BIN(24);
        goto END_IMD;
    }

    double A_f1 = fft_mag_idm[bin_f1];
    double A_f2 = fft_mag_idm[bin_f2];
    double A_sum = fft_mag_idm[bin_sum];
    double A_diff = fft_mag_idm[bin_diff];

```

```

double A_imd = (A_sum > A_diff) ? A_sum : A_diff;
double A_fund = (A_f1 > A_f2) ? A_f1 : A_f2;
double IMD_db = 20.0 * log10(A_imd / A_fund);

// ----- 10. 判断 -----
if (IMD_db > -41.0) { // 超限
    SHOW_RESULT("IMD", IMD_db, "dB", -41.0, No_LoLimit);
    BIN(25);
}
else{
    SHOW_RESULT("IMD", IMD_db, "dB", -41.0, No_LoLimit);
}

END_IMD:
}

double pcm_to_voltage_rms(unsigned char tp3057_byte)
{
    // 1. 偶数位反转, 恢复标准 A-law
    unsigned char alaw_std = tp3057_byte ^ 0x55;

    // 2. A-law 解码 (标准 G.711 算法)
    short linear = 0;
    unsigned char sign = alaw_std & 0x80;
    unsigned char magnitude = alaw_std & 0x7F;
    unsigned char segment = (magnitude >> 4) & 0x07;
    unsigned char step = magnitude & 0x0F;
    if (segment == 0) {
        linear = (step << 4) + 8;
    } else {
        linear = ((0x10 | step) << (segment + 2)) + 8;
    }
    if (sign) linear = -linear;

    // 3. 转换为峰值电压
    double vpeak = (double)linear * (VPEAK_FS / LINEAR_MAX);
    // 4. 转换为有效值 (正弦波)
    return vpeak / sqrt(2.0);
}

```

```

// 捕获多个 PCM 字节（从图形 start_idx 中，从 offset 周期开始提取）
// 返回实际捕获的字节数，数据存入 pcm_bytes 数组（至少 num_bytes 长度）
int capture_multi_pcm_bytes(int start_idx, int dx_channel, int
offset,
                           unsigned char* pcm_bytes, int num_bytes)
{
    BYTE* fail_bits = NULL;
    int depth = 0;
    fail_bits = GET_PATTERN_RESULT(start_idx, dx_channel, &depth);
    if (fail_bits == NULL || depth < offset + num_bytes * 9) {
        return 0;    // 数据不足
    }
    for (int byte = 0; byte < num_bytes; byte++) {
        unsigned char pcm_byte = 0;
        for (int bit = 0; bit < 8; bit++) {
            int cycle = offset + 1 + byte*256 + bit;    // 第五字节为 fsx
置 1, offset 为 4, 每个字节占用 256 个周期, fsx-8khz
            int byte_ofs = cycle / 8;
            int bit_ofs = cycle % 8;
            int level = (fail_bits[byte_ofs] >> bit_ofs) & 1;
            pcm_byte |= (level << (7 - bit));
        }
        pcm_bytes[byte] = pcm_byte;
    }
    return num_bytes;
}

void apply_hanning(double* data, int n) //加汉宁窗
{
    for (int i = 0; i < n; i++) {
        double w = 0.5 * (1 - cos(2 * M_PI * i / (n - 1)));
        data[i] *= w;
    }
}

```

9) 参考图形（具体图形见“向量图形.txt”）

索引号 7 号——抓取 PCM 码

6. 功能测试

- 1) **测试意义：**验证芯片基础收发功能和 A/D 功能是否正常。
- 2) **测试原理：**通过满幅运行传输接收功能
- 3) **测试条件：**短接模拟反相输入和 GSX，连接模拟正向输入至 VS1，对模拟正向输入施加 2.5V 偏置电压，运行图形 0（BOOL）
- 4) **被测引脚：**DX、VFRO
- 5) **测试步骤：**K1 和 K2 施加 5V 电压，等待 10ms，对模拟正向输入施加 2.5V 偏置电压，等待 10ms，运行图形 0（BOOL），等待 10ms 测量模拟输出，应为 $2.492V \pm 5\%$
- 6) **使用 ATE 资源：**K、VS、DVM
- 7) **参考代码**（具体代码见“程序代码.txt”）

```
//FUN-TEST
SETUP();

SET_RELAY("1,2");
Delay(10);
SET_VS1(2.5,V);
Delay(10);
RUN_PATTERN("FUNC_T",0,1,0,0);
Delay(10);
double Vout=DVM_MEASURE(1,3,V,10);
if (Vout <= 2.7 || Vout >= 2.3) {
    SHOW_RESULT("FUNC_R", Vout, "V", 2.7, 2.3);
}
else{
    SHOW_RESULT("FUNC_R", Vout, "V", 2.7, 2.3);
    BIN(26);
}
CLEAR_RELAY("1,2");
```

7. AI 工具使用声明

(a) 使用的 AI 工具名称、版本

DeepSeek-R1

(b) 使用场景与用途

- 辅助文献调研与资料梳理
- 辅助撰写技术文档引言、章节概述及总结
- 对部分代码进行格式优化与注释生成
- 文档文字润色与表述优化

(c) AI 生成内容占比

- 文字内容：约 15%
- 代码内容：约 10%

8. 参考文献

- [1] QY RESEARCH .Global Semiconductor Test Services Market Outlook, In-Depth Analysis & Forecast to 2032[R/OL].(2026-1-31).<https://www.qyresearch.com/reports/5806431/semiconductor-test-services>
- [2] Fortune Business Insights. 半导体测试设备市场规模、份额和行业分析，按产品类型（半导体自动测试设备 (ATE)、老化系统、处理设备、探针设备等）、技术（模拟测试、数字测试、混合信号测试、射频测试和功率半导体测试）、最终用途行业（半导体制造、消费电子、汽车、军事和国防、IT 和电信等）以及区域预测，2026 年至 2034 年[R/OL].(2026-4-20).
<https://www.fortunebusinessinsights.com/zh/semiconductor-test-equipment-market-113809>
- [3] 湖南贝哲斯信息咨询有限公司. 2025 年全球及中国自动化测试设备 (ATE)市场竞争情况与发展趋势分析报告[R/OL].(2025-10-15).<https://dxpress.gelonghui.com/p/2994772>
- [4] Fortune Business Insights.自动化测试设备市场规模、份额和行业分析，按设备类型（集成电路、硬盘驱动器、印刷电路板和其他（线路可更换单元））、按应用（汽车和运输、消费电子产品、电信、医疗保健、半导体制造和其他（军事、公用事业））以及区域预测，2026-2034 [R/OL].(2026-4-20).<https://www.fortunebusinessinsights.com/zh/automated-test-equipment-market-110360>
- [5] 湖南贝哲斯信息咨询有限公司.2025 年半导体自动测试设备 ATE 行业规模、竞争情况及增长趋势分析报告[R/OL].(2025-09-19).<https://dxpress.gelonghui.com/p/2868055>
- [6] 华创证券.半导体测试设备行业深度研究报告：算力迭代与先进封装重塑价值，国产测试设备步入替代加速期[J/OL].(2026-1-18).
https://m.sohu.com/a/977148982_122020073/?pvid=000115_3w_a&scm=10001.325_13-325_13.0.0-0-0-0-0.5_1334&spm=sm pc.channel_248.block3_308_NDdFbm_1_fd.1.17686686368

37EJHtlyL_324

- [7] 证星财报摘要. 华峰测控（688200）2025 年半年度管理层讨论与分析
[EB/OL].(2025-9-3).<https://wap.stockstar.com/detail/RB20250903000018396>